

## 2.1.1 – 2.1.3 Trace Tables vs Stepwise Reasoning

ٹریس ٹیبل الگورتھم کا کاغذی اجرا (Dry Run) ہے۔

CS XII

Chapter 2: Computational Thinking &amp; Algorithms

Lecture 02 of 16

★ Golden

### 2.1.1 – 2.1.3 Trace Tables vs Stepwise Reasoning

★ Golden

#### LEARN IT

A **trace table** is a technique used to monitor how the values of variables change during the execution of an algorithm. It is a **paper execution** of the code — a manual walkthrough, also called a **dry run**. You act as the computer and fill in one row after another.

#### Purpose of trace tables

- **Dry running:** execute the algorithm line by line by hand.
- **Debugging:** find logical errors where the algorithm runs but the output is wrong.
- **Validation:** check that the algorithm does what it is intended to do.

#### Components of a trace table

- **Line / step number** — which line is being executed.
- **Variables** — one column for every variable.
- **Input / output** — user inputs and printed results.
- **Condition check** — True/False for IF, WHILE, FOR, and similar tests.

## WORKED EXAMPLE

**Worked example: soldiers who meet the height rule.** To qualify for training, a soldier's height must be between **1.57 and 1.91 metres, inclusive**.

Inputs: all soldiers' heights. Output: the count of eligible soldiers.

Step 1: Start

Step 2: SET Total = 0

Step 3: Set AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]

Step 4: FOR EACH height FROM AllHeights DO

Step 5: IF height  $\geq$  1.57 AND height  $\leq$  1.91 THEN

Step 6: SET Total = Total + 1

Step 7: END IF

Step 8: END FOR EACH

Step 9: Print Total

Step 10: End

Inclusive means the two boundary values **count**: 1.57 and 1.91 are eligible.

| Step | Total | Height | AllHeights (input)           | Height between 1.57 and 1.91? | Output |
|------|-------|--------|------------------------------|-------------------------------|--------|
| 2    | 0     |        |                              |                               |        |
| 3    | 0     |        | 1.91, 2.01, 1.87, 1.57, 1.48 |                               |        |
| 4    | 0     | 1.91   |                              |                               |        |
| 5    | 0     | 1.91   |                              | True                          |        |
| 6    | 1     | 1.91   |                              |                               |        |
| 4    | 1     | 2.01   |                              |                               |        |
| 5    | 1     | 2.01   |                              | False (skip Step 6)           |        |
| 4    | 1     | 1.87   |                              |                               |        |
| 5    | 1     | 1.87   |                              | True                          |        |
| 6    | 2     | 1.87   |                              |                               |        |
| 4    | 2     | 1.57   |                              |                               |        |
| 5    | 2     | 1.57   |                              | True (boundary)               |        |

|   |   |      |  |                     |   |
|---|---|------|--|---------------------|---|
| 6 | 3 | 1.57 |  |                     |   |
| 4 | 3 | 1.48 |  |                     |   |
| 5 | 3 | 1.48 |  | False (skip Step 6) |   |
| 9 | 3 |      |  |                     | 3 |

Eligible: 1.91, 1.87, 1.57. Not eligible: 2.01 (above 1.91) and 1.48 (below 1.57). Printed Total = 3.

```
AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]
Total = 0
for height in AllHeights:
    if height ≥ 1.57 and height ≤ 1.91:
        Total = Total + 1
print(Total)
```

OUTPUT  
3

#### REMEMBER IT

**How to fill a trace table without getting lost.** Write a new row every time a variable **changes** or a condition is **tested**. Copy unchanged values into the new row so you never guess. Circle the final output.

Mnemonic: **V-I-C-O** — **V**ariables, **I**ntermediate, **C**ondition, **O**utput. If a column is missing, the table is incomplete.

#### COMMON MISTAKE

**Mistake 1:** treating 1.57 or 1.91 as out of range. The words “inclusive” and the operators  $\geq$  and  $\leq$  mean the boundaries **are allowed**.

**Mistake 2:** updating Total on a False condition. If the IF is False, skip the SET line and leave Total unchanged.

**Mistake 3:** writing only the last Total. Examiners award marks for the **changing values in the table**, not only for the printed 3.

ٹریس ٹیبل الگورتھم کا کاغذی اجرا (Dry Run) ہے۔ ہر قطار میں لکیر نمبر، متغیرات کی نئی قیمتیں، ان پٹ / آؤٹ پٹ اور شرط کا نتیجہ (True/False) لکھا جاتا ہے۔ فوجیوں کی مثال میں قد 1.57 سے 1.91 میٹر تک ہونا چاہیے، حدود سمیت؛ پانچ قدوں میں سے تین اہل ہیں اس لیے آؤٹ پٹ 3 ہے۔

#### EXAM TIP

Very common long question: "Dry-run the given algorithm using a trace table and state the output." Draw columns for **every** variable plus a condition column. Show the loop once per item. Boundary values ( $\geq$ ,  $\leq$ , empty list) are favourite traps. Short questions ask: purpose of a trace table (dry run / debug / validate) and its components (line, variables, I/O, condition).

#### CHECK YOURSELF

1. List the four usual columns / components of a trace table.
2. In the soldier example, why is height 1.57 counted but 1.48 not counted?
3. What is printed, and how many times is Step 6 executed?

Answers: 1. Line/step number, variables, input/output, condition check. 2. 1.57 is the lower boundary (inclusive, so True); 1.48 is below 1.57 (False). 3. 3 is printed; Step 6 runs three times (for 1.91, 1.87 and 1.57).

#### LEARN IT

**Stepwise reasoning** (also called **logic verification**) breaks an algorithm into logical sections and checks the **state** of the algorithm at each stage. A **state** is a snapshot of what is true about the variables at that moment (for example, "Sum currently holds the total of the numbers seen so far").

You do **not** fill a row for every line. You think through the logic:

1. Analyse each step of the algorithm.
2. Check: does it **start in a valid state**?
3. Does it **handle all possible cases**?
4. Are the **conditions correct**?
5. If the loop is correct at the **start** of an iteration, does it remain correct at the **end**?

## WORKED EXAMPLE

### Worked example: sum of the numbers from 1 to N.

Step 1: Start

Step 2: Input N

Step 3: SET Sum = 0

Step 4: For i = 1 to N

Step 5: Sum = Sum + i

Step 6: END For

Step 7: Print Sum

Step 8: End

### Stepwise reasoning breakdown

| Check            | What you verify                                                                |
|------------------|--------------------------------------------------------------------------------|
| Initial state    | Sum = 0 before the loop. Valid: nothing has been added yet.                    |
| Loop boundary    | The loop runs exactly from i = 1 to i = N. No missing number, no extra number. |
| Iteration        | Each i is added once: after k iterations, Sum = 1+2+...+k.                     |
| Final state      | When the loop ends, Sum holds 1+2+...+N, which is the required total.          |
| Edge case N = 0  | The loop body never runs, Sum stays 0. Correct (the sum of no numbers is 0).   |
| Check case N = 5 | Sum becomes 15, which matches $5 \times 6 / 2 = 15$ .                          |

```
N = 5
Sum = 0
for i in range(1, N + 1):
    Sum = Sum + i
print(Sum)
```

OUTPUT  
15

```
N = 0
Sum = 0
for i in range(1, N + 1):
    Sum = Sum + i
print(Sum)
```

**REMEMBER IT**

Think of stepwise reasoning as a **teacher checking a proof**, not as a cashier counting coins one by one. You ask “is this block always true?” instead of “what is Sum when  $i = 17$ ?”.

**COMMON MISTAKE**

**Mistake:** rewriting the whole trace table and calling it stepwise reasoning.

**Fix:** write short comments about **state** (start, loop, finish, edges). If you listed every  $i$  from 1 to 1000, you used the wrong method.

**اردو میں سمجھیں**

مرحلہ وار استدلال میں الگورتھم کو منطقی حصوں میں توڑ کر ہر مرحلے پر اس کی حالت (State) دیکھی جاتی ہے: کیا شروع درست ہے، کیا تمام صورتیں سنبھلی ہیں، کیا شرط صحیح ہے، اور کیا لوپ کے ہر چکر کے بعد بھی نتیجہ درست رہتا ہے۔ ایک سے  $N$  تک جمع کی مثال میں Sum شروع میں 0 ہے، لوپ 1 سے  $N$  تک چلتا ہے، اور  $N=0$  پر جواب 0 ہی رہتا ہے۔

**CHECK YOURSELF**

1. What other name is given to stepwise reasoning?
2. For the sum algorithm, why is  $\text{Sum} = 0$  a valid start?
3. What happens when  $N = 0$ , and is that correct?

Answers: 1. Logic verification. 2. Because no numbers have been added yet, so the running total should be zero. 3. The loop does not run and Sum remains 0, which is the correct sum of no numbers.

**LEARN IT**

Both methods verify correctness, but they fit different situations depending on the **complexity and length** of the algorithm.

| Feature  | Trace table                                                                       | Stepwise reasoning                                                    |
|----------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Best for | Small algorithm blocks, complex loops, and beginners learning to track variables. | Large systems, architectural logic, and abstract mathematical proofs. |

|                 |                                                                                |                                                                 |
|-----------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Effort required | High. Very tedious and impractical if a loop runs 1,000 times.                 | Moderate. Needs strong logical deduction and abstract thinking. |
| Primary goal    | Catching syntax misinterpretations or specific arithmetic errors line by line. | Catching conceptual or structural flaws in the overall design.  |
| Level           | <b>Micro-level</b> (line by line).<br>Checks exact values.                     | <b>Macro-level</b> (logic blocks).<br>Checks overall state.     |

#### Trace table

Micro-level  
Line by line  
Exact values

#### Stepwise reasoning

Macro-level  
Logic blocks  
Overall state

Trace tables zoom in on values; stepwise reasoning zooms out on design.

#### WORKED EXAMPLE

##### Worked example: which method would you pick?

1. **A 12-line loop that adds five heights.** Use a **trace table**. The loop is short, you are a beginner, and you need the exact Total.
2. **A billing system with 40 functions.** Use **stepwise reasoning**. You cannot fill 10,000 rows. You check each module's start state, end state and edge cases.
3. **A loop from  $i = 1$  to 1,000 that should produce  $\text{Sum} = 500500$ .** Do **not** write 1,000 rows. Argue: "Sum starts at 0; each  $i$  is added once; final  $\text{Sum} = n(n+1)/2 = 1000 \times 1001/2 = 500500$ ." That is stepwise reasoning. Then dry-run a tiny  $N$  (say 5) with a trace table if you want extra confidence.

#### REMEMBER IT

**Micro vs macro.** Trace table = microscope (every cell). Stepwise reasoning = map (whole city). Beginners start with the microscope; designers need the map.

#### COMMON MISTAKE

**Mistake:** saying one method is "better" in every question.

**Fix:** they have different jobs. For a short dry-run in the exam, draw the table. For "explain why the algorithm is correct for any  $N$ ", write stepwise reasoning.

ٹریس ٹیبل خرد بینی (Micro) سطح پر کام کرتا ہے: ہر لائن اور ہر قیمت۔ مرحلہ وار استدلال بڑی (Macro) سطح پر ڈیزائن اور حالت چیک کرتا ہے۔ چھوٹے لوپ اور مبتدیوں کے لیے ٹیبل بہتر ہے؛ ہزار بار چلنے والے لوپ یا بڑے سسٹم کے لیے استدلال بہتر ہے۔

#### EXAM TIP

Differentiate questions are almost guaranteed: "Compare a trace table with stepwise reasoning." Write a **four-row table** (best for, effort, primary goal, micro/macro). Add one sentence of advice: table for small loops and exact values; reasoning for large systems and proofs. Do not write "trace table is always better".

#### CHECK YOURSELF

1. Which method is micro-level and checks exact values?
2. Why is a trace table a poor choice if a loop runs 1,000 times?
3. Which method is better for catching a structural flaw in a large design?

Answers: 1. Trace table. 2. It is very tedious and impractical (too many rows). 3. Stepwise reasoning (macro-level, conceptual / structural flaws).

## Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

**MCQ 1.** A trace table is:

- a. A dry-run grid of variables vs steps
- b. A database table
- c. A pie
- d. A Figma frame

**MCQ 2.** Stepwise reasoning checks:

- a. The state at logical sections, not every micro-step
- b. Only Big O
- c. Only colours
- d. Only SQL

**MCQ 3.** Use a trace table when:

- a. Few variables, you must not miss an assignment
- b. The algorithm is a huge system diagram only
- c. You need UX quotes
- d. You need a histogram

**MCQ 4.** Use stepwise when:

- a. The logic is chunky (phases) rather than many tiny vars
- b.  $n=2$  and 8 columns of  $i$
- c. You sort visually
- d. You draw OSI

**MCQ 5.** Both methods aim at:

- a. Correctness
- b. Careers
- c. Fonts
- d. TCP windows

**MCQ 6.** A soldier-eligibility trace would track:

- a. Each input and the count so far
- b. CSS padding
- c. NAND groups
- d. Sprint points

## Extra short questions

1. Define trace table vs stepwise reasoning.
2. When each is better (book comparison).
3. What must every trace column include?
4. A common student error.

## Extra long question

1. Build a full trace table for: count how many of 3 heights are  $\geq 5.5$ . Then write three stepwise sentences. ★ Golden.

## Extra answer key

**MCQs:** 1. A 2. A 3. A 4. A 5. A 6. A

### Short

1. Trace: column per variable, row per step. Stepwise: argue each phase's post-condition.
2. Trace: loops with a few vars. Stepwise: long algorithms with clear sections.
3. Changing variables and the output/decision.
4. Updating i in the table but forgetting the accumulator.

### Long

1. Columns: i, height, count. Rows for each soldier. Stepwise: init 0; for each, if tall count++; after loop count is the answer. Compare the two explanations.

---

Next lecture: 2.2 Evaluating the Clarity of an Algorithm